РОЗРОБКА ЗАСТОСУНКІВ ЗІ ШТУЧНИМ ІНТЕЛЕКТОМ

Практична робота №11. Спостережуваність і контроль ресурсів AI-компонента

Мета: Зробити AI-сервіс із ПР10 таким, що про нього можна відповісти на питання «що сталося, скільки це коштувало і чому так довго»: додати структурований журнал з ідентифікатором запиту, моделлю, версією prompt, тривалістю, статусом, видом збою, токенами й вартістю кожного виклику; реалізувати кеш відповідей і денний ліміт витрат; за журналом знайти повільні, повторні й невдалі виклики та порівняти роботу застосунку з кешем і без нього.

Зміст

1. Теоретичні відомості
2. Завдання до виконання
3. Критерії приймання
4. Контрольні запитання
5. Рекомендована література

1. Теоретичні відомості

Помічник оператора з ПР10 пропрацював два тижні, і до вас прийшли з питаннями. Керівник підтримки: скільки коштували чернетки минулого тижня і чому у вівторок утричі дорожче, ніж у понеділок? Оператори: учора близько 14:00 усе було повільно — це провайдер чи наша черга? Той самий керівник: чернетка для звернення Петренка тричі завершилася збоєм, а потім раптом вийшла — що це було? Фінансовий відділ: у ключа місячний бюджет, і застосунок не має права його перевищити — хто це гарантує? Застосунок із ПР10 не відповість на жодне. Журнал подій чернетки живе в браузері оператора, реєстр — у пам'яті процесу до першого перезапуску, а журнал uvicorn каже лише `POST /api/drafts 202`. Відповісти можна, якщо застосунок лишає слід від кожного виклику в такому вигляді, у якому його потім можна запитувати, і якщо між шлюзом і провайдером стоять дві перевірки: чи не відповідали ми на це саме питання щойно і чи є ще на це гроші.

Сценарій той самий: чернетки відповідей клієнтам «Сузір'я». Код ПР10 — відправна точка; у цій роботі він не переписується, а доповнюється.

Опора, щоб виконувати роботу свідомо.



Три сигнали. Про систему, яка працює, дізнаються з трьох джерел. Журнал (logs) — записи про події: що сталося, коли, з якими подробицями. Метрики (metrics) — числа в часі: скільки запитів за хвилину, яка частка збоїв, який 95-й процентиль тривалості. Траси (traces) — ланцюжки записів, зв'язані одним ідентифікатором: шлях одного запиту крізь усі рівні. У великих системах це три окремі інструменти; у застосунку з одного процесу досить журналу, записаного так, щоб із нього можна було обчислити метрики й зібрати трасу. Саме це тут і робиться: один файл, одна подія на рядок, спільні ідентифікатори. Коли сервісів стане кілька, ті самі поля переїдуть у стандартну телеметрію — OpenTelemetry має окремі домовленості про назви полів для викликів генеративних моделей: назва моделі, кількість токенів на вході й виході, причина зупинки.

Структурований журнал. Рядок `Draft 7f3e done in 4.1s (openai, 3 attempts)` зрозумілий людині й марний для запитів: щоб знайти всі чернетки довші за 10 с, його треба розбирати регулярним виразом, а наступного тижня формат зміниться. Структурований запис — це подія з полями: `{"ts": …, "event": "draft.done", "draft_id": "7f3e", "duration_ms": 4100, "provider": "openai", "attempts": 3}`. Один запис — одна подія — один рядок JSON (формат JSON lines). Файл такого вигляду читається десятьма рядками Python, завантажується в будь-яку систему журналів і не потребує розбору. Рівні лишаються: `INFO` — штатні події, `WARNING` — те, що спрацювало, але не мало б (друга спроба, запасний варіант, 80 % бюджету), `ERROR` — збій після всіх спроб. Які події записувати — рішення на межі між «замало» і «забагато». Замало — коли не можна відповісти на питання з першого абзацу; забагато — коли кожен фрагмент потоку стає записом і файл росте на мегабайти за годину. Орієнтир: подія на кожну зміну стану чернетки і на кожне звертання до провайдера, а не на кожен байт. Окреме правило — що в журнал не потрапляє ніколи: ключ доступу, текст звернення клієнта і факти з іменами й номерами замовлень (це персональні дані, а журнал читають і зберігають не ті, хто має до них доступ),

повний prompt. Замість них — ідентифікатор звернення, довжини, хеш, версія. Слід стека в журналі доречний, на сторінці — ні.

Ідентифікатори й кореляція. Чернетка — це кілька записів з різних рівнів і з різних потоків виконання, і щоб зібрати їх докупи, кожен запис має нести ті самі ідентифікатори. Ідентифікатор запиту виникає на межі застосунку: веб-рівень генерує його для кожного HTTP-запиту, кладе в контекст і повертає клієнтові в заголовку `X-Request-ID` та в тілі помилки. Оператор, який скаржиться «не працює», називає цей ідентифікатор, і ви знаходите всі записи за секунду. Ідентифікатор чернетки живе довше за HTTP-запит: замовлення, події, скасування, повтор — усе це різні запити до однієї чернетки. Номер спроби розрізняє звертання до провайдера всередині однієї чернетки. Траса чернетки — це ланцюжок: `http.request` → `draft.created` → `llm.attempt` (таймаут) → `llm.attempt` (успіх) → `draft.done`. Технічна пастка: контекстна змінна (`contextvars`) з ідентифікатором запиту діє в потоці, який обробляє HTTP-запит; фонова задача з ПР10 виконується в іншому потоці, і там змінна порожня. Ідентифікатори або передають явно, або копіюють контекст у новий потік.

Що фіксувати для виклику моделі. Запис про одне звертання до провайдера має відповісти на всі питання, які вам поставлять, і мати ті поля, які сторінка чи аналіз потребуватимуть, не перечитуючи код. Звідки й куди: ідентифікатори запиту й чернетки, клієнт, звернення. Хто відповідав: провайдер і модель, яка відповіла насправді (у запасному варіанті вона інша, ніж замовлена), версія prompt, параметри генерації. Скільки тривало: очікування в черзі до початку, час до першого фрагмента, разом; саме поділ показує, де повільно — у нашій черзі чи в провайдера. Чим закінчилося: статус, вид збою, номер спроби, чи це запасний варіант, чи з кешу. Скільки коштувало: токени на вході, на виході, кешовані провайдером, і вартість у грошах. Це той самий набір, який показують панелі комерційних платформ спостережуваності для LLM; тут він збирається вручну, щоб було зрозуміло, звідки береться кожне число.

Версія prompt. Інструкція для моделі — частина коду, і вона змінюється: додали речення про тон, переставили факти, змінили формат. Відповіді змінюються разом із нею, а через тиждень ніхто не пам'ятає, коли саме. Тому кожен виклик записують із версією prompt: рядок, який змінюють разом з інструкцією (`drafts-v3`), або хеш тексту інструкції, який змінюється сам. Ручна версія читабельна, хеш — чесний, бо його не можна забути підняти; можна тримати обидва. Версія потрібна трьом речам: журналу (щоб «у вівторок стало гірше» зіставити з «у вівторок вийшла v3»), кешу (стара відповідь на нову інструкцію — не відповідь) і набору для оцінювання, до якого дійде ПР12. Те саме з моделлю: назва в `.env` — побажання, а що відповіло насправді — у результаті провайдера; за однією назвою провайдери підставляють датовані версії і час від часу міняють їх.

Вартість. Провайдер тарифікує токени: вхідні за однією ціною, вихідні за іншою, зазвичай у кілька разів вищою; вхідні, які провайдер узяв зі свого кешу, — дешевше. Вартість виклику — токени, помножені на ціни, а кількість токенів провайдер повертає в полі `usage` відповіді; у режимі streaming — в останньому фрагменті, і лише якщо його попросили. Якщо `usage` не прийшов (потік обірвано, провайдер не підтримує), вартість оцінюють за довжиною тексту й позначають як оцінку, а не вигадують точне число. Безкоштовний рівень провайдера нічого не змінює: рахунок нульовий, але витрати рахують так, ніби платять, інакше бюджет і порівняння нема з чим робити, а безкоштовний рівень закінчується. Числа про вартість кажуть про архітектуру більше, ніж здається: чернетка коштує стільки-то токенів, із них 80 % — інструкція і

факти на вході; `max_tokens` обмежує найгірший випадок на виході; спроба, перервана таймаутом після половини тексту, усе одно оплачена; п'ять однакових замовлень за хвилину — п'ять оплат. Вартість чернетки, помножена на чернетки за день і на операторів, — рядок у бюджеті, і саме його просять оцінити.

Кешування. Повтори беруться звідусіль: оператор натиснув «Повторити», нічого не змінивши; натиснув п'ять разів, бо здалося, що зависло; двоє операторів взялися за одне звернення. Кеш відповідей повертає готовий результат, не звертаючись до провайдера. Точний кеш (exact cache) працює за ключем — хешем усього, що впливає на відповідь: провайдер, модель, версія prompt, повідомлення, температура, `max_tokens`; і нічого з того, що не впливає: ідентифікатор клієнта, номер чернетки, час. Значення — результат цілком. Термін життя обмежений: факти звернення можуть змінитися. Влучання треба позначати: оператор має бачити «з кешу», бюджет — не списувати, а сторінка, яка чекає на фрагменти, — отримати текст, навіть якщо він прийшов одразу. Кеш доводиться обходити: «Повторити» — це прохання про іншу чернетку, і за температури вище нуля інша можлива, тож прапорець `fresh` пропускає кеш. П'ять однакових замовлень одночасно — особливий випадок: усі п'ять промахнуться, бо перше ще не завершилося; об'єднати їх — щоб друге дочекалося першого — окреме рішення. Зміна версії prompt змінює ключ і робить старі записи недосяжними без окремого очищення. Кеш у пам'яті зникає з перезапуском; кеш на диску — ні, але потребує обмеження розміру. Семантичний кеш — коли схожі, а не однакові питання отримують ту саму відповідь за близькістю ембедінгів — заощаджує більше й ризикує більше: два звернення з різними номерами замовлень семантично майже однакові, а відповідь на чуже замовлення — помилка. Для чернеток із фактами він не підходить, для довідкових питань у ПР6 — міг би. Окремо від усього цього стоїть кешування prompt на боці провайдера: він упізнає однаковий початок запиту (інструкцію) і бере вхідні токени дешевше; виклик при цьому відбувається, модель генерує знову, і видно це лише в полі `usage` як кешовані токени.

Ліміт використання. Обмеження частоти з ПР10 рахує запити за хвилину; воно не завадить витратити місячний бюджет за три дні рівномірним потоком. Ліміт використання рахує гроші або токени за період: на весь застосунок і, за потреби, на клієнта. Ліній дві. М'яка — попередження: на 80 % бюджету оператор бачить, що він закінчується, журнал отримує `WARNING`, хтось дізнається раніше, ніж усе зупиниться. Жорстка — відмова: контрольована помилка з видом `budget`, повідомленням для людини і часом, коли ліміт відновиться; повторювати її немає сенсу до скидання. Який HTTP-статус для «гроші скінчилися» — рішення: 429 з `Retry-After` до півночі, 402, 503 — у кожного є аргументи. Що працює, коли бюджет вичерпано: влучання в кеш безкоштовні, тож їх можна віддавати; запасний варіант із підставним провайдером безкоштовний теж, але чи потрібна операторові вигадана чернетка — питання до нього, а не до коду. Облік чесний лише тоді, коли сходиться з рахунком провайдера: перевірка перед викликом рахує за відомими витратами, а вартість виклику відома після нього; два одночасні виклики обидва пройдуть перевірку на останні копійки. Лічильник у пам'яті обнулюється з перезапуском — і перезапуск «поповнює» бюджет; тому стан лічильника живе на диску. Межа доби залежить від часового поясу: день провайдера й день застосунку можуть не збігатися.

Аналіз журналів. Питання перетворюються на запити до файлу. Повільні: не середнє, а розподіл — медіана й 95-й процентиль; «повільно» означає «довше за звичайне», і звичайне треба спершу виміряти; розріз за провайдером, моделлю, годиною; а головне — де пройшов час: у черзі шлюзу, до першого фрагмента чи в генерації. Невдалі: за видом збою, за часом, за

моделлю; чи допомогли повтори — яка частка чернеток з невдалою першою спробою завершилася успіхом. Повторні: той самий ключ кешу кілька разів за короткий час (це кандидати на кеш і на запитання «чому»), та сама чернетка з кількома спробами (це повтори шлюзу), один клієнт із серіями замовлень (це «надіслати 5 поспіль»). Витрати: за день, за модель, за клієнта; і окремо — вартість спроб, які закінчилися збоєм: заплачено ні за що. Інструмент — тридцять рядків Python над JSON lines або `jq`; коли записів стануть мільйони, їх завантажать у систему журналів з панелями, але питання лишаться ті самі, і поля для них мають бути в записах уже зараз.

Порівняння з кешем і без. Один і той самий потік замовлень прогнати двічі — з вимкненим кешем і з увімкненим — і порівняти: звертань до провайдера, токенів, вартості, тривалості (медіана й 95-й процентиль), частки влучань. Кеш допомагає рівно настільки, наскільки в потоці є повтори: на синтетичному потоці з повторами він покаже економію, якої на справжньому навантаженні може не бути. Тому число, яке має значення, — частка влучань на реальних замовленнях за тиждень, і його дає той самий журнал. Там же видно шкоду: чернетка з кешу, коли оператор чекав на іншу, застарілі факти, однаковий текст двом операторам.

2. Завдання до виконання

Помічник оператора «Сузір'я» з ПР10 працює. Доповніть його так, щоб за його журналом можна було відповісти на питання «що сталося з цією чернеткою», «чому вчора було повільно», «скільки коштував тиждень» і «що ми оплатили двічі», а витрати були обмежені згори.

Застосунок має:

- вести **структурований журнал** у форматі JSON lines: одна подія — один запис; для кожного звертання до провайдера зафіксовано ідентифікатор запиту й чернетки, клієнта, провайдера, модель, яка відповіла, **версію prompt**, номер спроби, час до першого фрагмента й разом, статус і вид збою, запасний варіант, токени на вході й виході та **вартість**; ключ, текст звернення і факти в журнал не потрапляють;
- присвоювати кожному HTTP-запиту **ідентифікатор**, повертати його в заголовку `X-Request-ID` і в тілі помилки, передавати у фонову задачу — так, щоб усі записи однієї чернетки можна було зібрати за ним;
- рахувати **вартість** кожного виклику за токенами й цінами з налаштувань, позначати оцінку, якщо провайдер не повернув `usage`; показувати вартість чернетки операторові;
- **кешувати** відповіді за ключем, що охоплює все, що впливає на відповідь, з обмеженим терміном життя; позначати результат з кешу; обходити кеш за прапорцем `fresh` із кнопки «Повторити»; вмикатися й вимикатися налаштуванням;
- тримати **денний ліміт витрат** на весь застосунок: попередження на заданій частці, відмова після вичерпання з контрольованою помилкою виду `budget` і часом відновлення; стан ліміту переживає перезапуск; оператор бачить витрачене й залишок;
- давати **аналіз журналу** скриптом: найповільніші виклики й розподіл тривалості, невдалі за видом, повторні (той самий ключ, та сама чернетка), витрати за день і модель, частка влучань кешу.

Журнал, вартість, кеш і ліміт — кожне в **окремому модулі**; шлюз викликає їх, але не знає, як вони влаштовані; веб-рівень не знає про ціни й токени. Бекенд — FastAPI, як у ПР10.

Спосіб реалізації — на ваш розсуд (самостійно або скеровуючи AI-асистента), але ви маєте **розуміти й уміти пояснити** кожне рішення.

Матеріали. В архіві до цієї роботи, папка `pr11`: `requirements.txt`, скрипт перевірки середовища, зразок налаштувань, каркас застосунку, ті самі вісім звернень у `samples/`, папка `eval/` зі сценаріями, скриптом прогону замовлень через API (готовий) і заготовкою скрипта аналізу журналу. У каркасі готові: налаштування журналу у форматі JSON lines з ідентифікатором запиту в контексті (`telemetry.py`), веб-рівень із проміжним шаром, який присвоює ідентифікатор і повертає його в заголовку, ендпоінт стану витрат і сторінка, яка показує вартість, позначку «з кешу», бюджет і ідентифікатор запиту в помилках. Заготовки з переліком рішень: вартість (`costs.py`), кеш (`cache.py`), ліміт (`budget.py`), аналіз журналу (`eval/analyze.py`). Модулі ПР10 — провайдери, шлюз, фонові задачі, обмеження частоти, складання запиту — в каркасі лишені заготовками з позначкою «Заготовка з ПР10»: замініть їх своїми й допишіть те, що в кожному перелічено для ПР11.

Де вести роботу. Розпакуйте архів у корінь власного репозиторію курсу, поруч із папками попередніх робіт. Скопіюйте свої модулі з `pr10/app/` поверх заготовок; ключ доступу — той самий, перенесіть у `.env`. Журнал і стан кешу та бюджету пишуться в `logs/` і `state/` — обидві папки виключено з репозиторію.

Орієнтири.

- Журнал — стандартний модуль `logging`; формат JSON lines і файл задано в `telemetry.py`. Поля записів передаються через `extra={...}` і потрапляють у рядок JSON як є; подія — у повідомленні (`log.info("llm.attempt", extra={...})`). Ідентифікатор запиту — контекстна змінна `contextvars.ContextVar`; у новий потік контекст не переходить, його копіюють (`contextvars.copy_context().run`) або передають ідентифікатор аргументом.
- `usage` у відповіді OpenAI-сумісного API: `prompt_tokens`, `completion_tokens`, `total_tokens`; кешовані провайдером вхідні — у `prompt_tokens_details.cached_tokens`, якщо є. У streaming `usage` приходить в останньому фрагменті лише з `stream_options={"include_usage": True}`; перевірте, чи підтримує це ваш провайдер, — скрипт перевірки середовища покаже.
- Ціни — у `.env` за мільйон токенів, окремо на вхід і вихід; значення зі зразка орієнтовні, чинні — на сторінці цін провайдера. Підставний провайдер за замовчуванням рахується за тими самими цінами, щоб бюджет можна було перевірити без ключа.
- Ключ кешу — `hashlib.sha256` від канонічного подання запиту (`json.dumps` з `sort_keys=True`). Сховище — словник у пам'яті або файл JSON у `state/`; термін життя — час запису плюс TTL.
- Стан ліміту — файл у `state/` з датою і сумою; запис атомарний, як у ПР9.
- Скрипт `eval/replay.py` замовляє чернетки через API з повторами за детермінованою послідовністю, чекає завершення кожної й підсумовує: статуси, влучання, спроби, вартість, тривалість. Один і той самий `--seed` дає один і той самий потік для прогону без кешу і з ним. Ліміт на клієнта з ПР10 на час прогону підніміть у `.env`.
- Аналіз — `eval/analyze.py`: читання файлу готове, розділи звіту — заготовки; поля, за якими вони працюють, — ваші.

Продумайте самостійно.

- Які події записувати, а які ні? Що станеться з файлом за день роботи десяти операторів, якщо записувати кожен фрагмент потоку? Чого не вистачить, якщо записувати лише завершення

чернетки?

- Що з журналу можна показати будь-кому, а що — ні? Де в ваших записах могли б опинитися ім'я клієнта, номер замовлення, текст звернення, ключ — і як ви гарантуєте, що їх там немає?
- Ідентифікатор запиту, ідентифікатор чернетки, номер спроби — що кожен із них зв'язує? Як ідентифікатор потрапляє у фоновий потік, у подію `error` для сторінки, у відповідь 429?
- Версія prompt — рядок чи хеш? Що станеться з кешем, журналом і порівнянням якості, коли ви зміните одне слово в інструкції й забудете підняти версію?
- Звідки береться вартість спроби, яка завершилася таймаутом після половини тексту? А скасованої? А з кешу? Що записати, коли `usage` не прийшов?
- Що входить у ключ кешу, а що — ні? Чи входить температура? Клієнт? Поточна дата? Що буде, якщо факти звернення змінилися, а ключ — ні?
- П'ять однакових замовлень за секунду: скільки звертань до провайдера з кешем? Чи варто, щоб друге чекало на перше, і скільки?
- «Повторити» обходить кеш. А «Скласти чернетку» для того самого звернення через хвилину? Через день? Чим це відрізняється для оператора?
- Влучання в кеш — усе одно фонова задача з подіями? Що бачить оператор: текст одразу, «з кешу», час 0 с? Чи списує воно бюджет і ліміт частоти?
- Перевірка бюджету — перед замовленням, перед кожною спробою чи перед кожним платним викликом? Що з двома одночасними викликами на останню копійку? Що з чернеткою, яка вже генерується, коли бюджет вичерпався посеред неї?
- Який HTTP-статус і яке повідомлення у відмови через бюджет? Що оператор має зробити далі? Чи вмикати запасний варіант, коли грошей немає?
- Що таке «повільно» у ваших числах? Медіана, 95-й процентиль, поріг у секундах? Де пройшов час у найповільніших — у черзі, до першого фрагмента, у генерації?
- Чи допомогли повтори з ПР10: скільки чернеток врятувала друга спроба і скільки коштували спроби, які не врятували нічого?
- Скільки заощадив кеш на прогоні з повторами — і скільки заощадить на справжніх замовленнях, де повторів може не бути? Яке число ви назвали б керівникові?

Перевірка. Прожене сценарії з `eval/README.md`: траса однієї чернетки за ідентифікатором; перевірка журналу на ключ і персональні дані; збої й повтори в журналі; повільні виклики й розподіл тривалості; прогін без кешу і з кешем на одному потоці замовлень; п'ять одночасних замовлень з кешем; обхід кешу з «Повторити»; зміна версії prompt; попередження і вичерпання бюджету з перезапуском; справжній провайдер щонайменше на десяти замовленнях із фактичним `usage`; вартість без `usage`. Для кожного сценарію зафіксуйте налаштування, записи журналу, які його підтверджують, числа зі скрипта аналізу та що бачив оператор. Висновки — у файл.

Заготовка для цього — папка `eval/` зі сценаріями, скриптом прогону, заготовкою аналізу й питаннями, на які відповісти у висновках.

3. Критерії приймання

- Журнал — у **форматі JSON lines**, одна подія на запис; записи звертань до провайдера містять ідентифікатори запиту й чернетки, клієнта, провайдера, модель, **версію prompt**,

спробу, час до першого фрагмента й разом, статус, вид збою, запасний варіант, токени й **вартість**.

- Кожен HTTP-запит має **ідентифікатор**, який повертається в заголовку `X-Request-ID` і в тілі помилки; усі записи однієї чернетки, включно з фоновими, можна зібрати за ідентифікаторами.
- У журналі **немає** ключа, тексту звернення, фактів з іменами й номерами, повного prompt; слід стека — лише в журналі, не на сторінці.
- **Вартість** рахується за токенами з `usage` і цінами з налаштувань; якщо `usage` немає — оцінка, позначена як оцінка; оператор бачить вартість чернетки.
- **Кеш** працює за ключем, що охоплює провайдера, модель, версію prompt, повідомлення й параметри генерації; має TTL; результат з кешу позначений для оператора й не списує бюджет; «Повторити» обходить кеш; кеш вмикається й вимикається налаштуванням.
- **Денний ліміт** діє на весь застосунок: попередження на заданій частці видно операторові й у журналі; після вичерпання — контрольована помилка виду `budget` з часом відновлення, не 500; стан ліміту переживає перезапуск.
- Скрипт аналізу за журналом показує **повільні** (розподіл, не лише середнє), **невдалі** за видом, **повторні** (ключ кешу, спроби однієї чернетки), **витрати** за день і модель, частку влучань кешу.
- Прогін одного потоку замовлень **без кешу і з кешем** виконано; порівняно звертання до провайдера, токени, вартість, тривалість; результат пояснено.
- Журнал, вартість, кеш і ліміт — в окремих модулях; шлюз викликає їх через кілька функцій і не знає їхньої будови; ціни, TTL, ліміт, файл журналу — з `.env`; ключ **не потрапив** ані в код, ані в репозиторій, ані в журнал.
- Усі сценарії з `eval/` прогнано, у тому числі на справжньому провайдері з фактичним `usage`; висновки зафіксовано.
- Ви можете **пояснити** кожне ухвалене рішення.

4. Контрольні запитання

1. Чим структурований журнал відрізняється від текстового і що він дає, чого не дає текстовий? Які події ви записуєте, а які свідомо ні?
2. Покажіть трасу однієї чернетки з трьома спробами: які записи, з яких модулів, за якими полями вони зв'язані? Як ідентифікатор запиту потрапив у фоновий потік?
3. Чого в вашому журналі немає й чому? Як оператор повідомляє про проблему так, щоб ви знайшли її за секунду, не маючи його тексту звернення?
4. Що таке версія prompt у вашому застосунку, як вона змінюється і на що впливає — журнал, кеш, оцінювання?
5. Як рахується вартість виклику? Звідки токени в streaming? Що ви записуєте, коли `usage` не прийшов, і чим оцінка відрізняється від факту в журналі й у бюджеті?
6. Що входить у ключ кешу, що — ні, і чому? Що робить `fresh`? Що станеться з п'ятьма однаковими замовленнями одночасно?
7. Чим кеш відповідей у вашому застосунку відрізняється від кешування prompt на боці провайдера і від семантичного кешу? Чому семантичний кеш не підходить для чернеток із

фактами?

8. Як влаштовано денний ліміт: де перевірка, де облік, де стан? Що з двома одночасними викликами на залишок бюджету і з перезапуском? Що бачить оператор після вичерпання?
9. Що показав аналіз журналу: які виклики найповільніші і де пройшов час; який вид збою найчастіший; чи окупилися повтори; скільки коштував день?
10. Що дав кеш на вашому прогоні — звертань, токенів, грошей, часу — і чому це число не можна напряму переносити на справжнє навантаження?

5. Рекомендована література

1. Huyen C. AI Engineering: Building Applications with Foundation Models. Sebastopol: O'Reilly Media, 2025. 496 p. ISBN 978-1-0981-6630-4.
2. Majors C., Fong-Jones L., Miranda G. Observability Engineering: Achieving Production Excellence. Sebastopol: O'Reilly Media, 2022. 318 p. ISBN 978-1-4920-7644-5.
3. Beyer B., Jones C., Petoff J., Murphy N. R. (eds.). Site Reliability Engineering: How Google Runs Production Systems. Sebastopol: O'Reilly Media, 2016. 552 p. ISBN 978-1-4919-2912-4. URL: https://sre.google/sre-book/table-of-contents/ (дата звернення: 20.10.2026).
4. Sigelman B. H., Barroso L. A., Burrows M. et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report dapper-2010-1. Mountain View: Google, 2010. URL: https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/ (дата звернення: 23.10.2026).
5. Bang F. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings // Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023). Singapore: ACL, 2023. P. 212–218.
6. Chen L., Zaharia M., Zou J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176. URL: https://arxiv.org/abs/2305.05176 (дата звернення: 19.10.2026).
7. Kleppmann M. Designing Data-Intensive Applications. Sebastopol: O'Reilly Media, 2017. 616 p. ISBN 978-1-4493-7332-0.
8. OpenTelemetry. Semantic Conventions for Generative AI Systems. URL: https://opentelemetry.io/docs/specs/semconv/gen-ai/ (дата звернення: 23.10.2026).
9. Python Documentation. logging — Logging facility for Python. URL: https://docs.python.org/3/library/logging.html (дата звернення: 23.10.2026).
10. Python Documentation. contextvars — Context Variables. URL: https://docs.python.org/3/library/contextvars.html (дата звернення: 23.10.2026).
11. OpenAI. Prompt caching — API guide. URL: https://platform.openai.com/docs/guides/prompt-caching (дата звернення: 23.10.2026).
12. Google AI for Developers. Gemini API — Pricing. URL: https://ai.google.dev/gemini-api/docs/pricing (дата звернення: 23.10.2026).
13. Google AI for Developers. Gemini API — Context caching. URL: https://ai.google.dev/gemini-api/docs/caching (дата звернення: 23.10.2026).
14. OpenAI. API Reference — Chat Completions. URL: https://platform.openai.com/docs/api-reference/chat (дата звернення: 01.09.2026).

15. FastAPI Documentation. URL: https://fastapi.tiangolo.com/ (дата звернення: 01.09.2026).